

STM32F407 NTSC Composite Video

Complete Mathematics for SPI DMA + Double Buffering

320 × 200 monochrome video engine

This document adapts the supplied first-principles NTSC mathematics to the actual project architecture: STM32F407 timer-based horizontal timing, SPI pixel streaming, DMA, a 320×200 1-bit framebuffer, and double buffering. The calculations are engineering design equations; final waveform values should be verified on the actual hardware with an oscilloscope and display.

1. NTSC Master Timing

Frame rate: $f_F \approx 29.97 \text{ Hz}$

Frame period: $T_F = 1 / 29.97 \approx 33.3667 \text{ ms}$

Lines per frame: $N_L = 525$

Horizontal frequency: $f_H = 525 \times 29.97 \approx 15,734.264 \text{ Hz}$

Horizontal period: $T_H = 1 / 15,734.264 \approx 63.5556 \text{ }\mu\text{s}$

Field period: $T_{\text{FIELD}} = 262.5 \times 63.5556 \text{ }\mu\text{s} \approx 16.6833 \text{ ms}$

Field rate: $f_{\text{FIELD}} \approx 59.94 \text{ Hz}$

2. Horizontal Line Budget

Interval	Approx. time	Purpose
Horizontal sync	4.7 μs	Sync level
Back porch	5.3 μs	Blanking / return
Active video	52.7 μs	Pixel data
Front porch	0.9 μs	Blanking
Total	$\approx 63.6 \text{ }\mu\text{s}$	One line

$$T_{\text{LINE}} = T_{\text{SYNC}} + T_{\text{BP}} + T_{\text{ACTIVE}} + T_{\text{FP}}$$

3. STM32F407 Timer Mathematics

For the common STM32F407 clock arrangement used in the supplied document: SYSCCLK = 168 MHz, PCLK1 = 42 MHz, and APB1 timers run at 84 MHz when the APB prescaler is greater than 1.

$$f_{\text{TIM}} = 84 \text{ MHz}$$

$$T_{\text{TICK}} = 1 / 84,000,000 = 11.90476 \text{ ns}$$

$$N_{\text{LINE}} = T_{\text{LINE}} \times f_{\text{TIM}} \approx 63.5556 \text{ }\mu\text{s} \times 84 \text{ MHz} \approx 5,339.999 \approx 5,340 \text{ counts}$$

$$\text{ARR} = N_{\text{LINE}} - 1 = 5,339$$

$$N_{\text{SYNC}} = 4.7 \text{ }\mu\text{s} \times 84 \text{ MHz} \approx 395 \text{ counts}$$

The timer establishes the 15.734 kHz line cadence. DMA does not replace this line timing; it supplies the pixel/sample bytes inside the active-video window.

4. 320 × 200 Monochrome Framebuffer

Pixels per byte = 8

Bytes per line = WIDTH / 8 = 320 / 8 = 40 bytes

Framebuffer size = 40 × 200 = 8,000 bytes

Double framebuffer memory = 2 × 8,000 = 16,000 bytes

Address of line y = framebuffer_base + y × 40

Each framebuffer byte contains eight horizontal monochrome pixels. A bit value of 0 represents black and 1 represents white, subject to the project's output polarity.

5. Pixel and SPI-DMA Mathematics

For the actual DMA design, 40 bytes are transmitted for each 320-pixel line. If SPI sends one byte containing eight pixels, then the SPI clock must provide eight serial bits per byte.

$N_BYTE = WIDTH / 8 = 40 \text{ bytes/line}$

$N_BIT = 40 \times 8 = 320 \text{ bits/line}$

$T_DMA_LINE = N_BYTE \times 8 / f_SPI$

$f_SPI = (N_BYTE \times 8) / T_ACTIVE$

Using $T_ACTIVE \approx 52.7 \mu s$:

$f_SPI = 320 / 52.7 \mu s \approx 6.072 \text{ MHz}$

$T_BIT \approx 1 / 6.072 \text{ MHz} \approx 164.7 \text{ ns}$

$T_BYTE = 8 \times T_BIT \approx 1.318 \mu s$

$40 \times 1.318 \mu s \approx 52.7 \mu s$

This is an ideal timing calculation. The real SPI clock should be selected from STM32F407 SPI clock divisors and then the actual active-video duration should be measured. DMA transfer time must fit entirely inside the intended active-video window.

6. Choosing a Real SPI Clock

The design equation is more useful than assuming a single magic SPI number:

$T_DMA = (WIDTH / 8) \times 8 / f_SPI$

For WIDTH = 320, $T_DMA = 320 / f_SPI$. Therefore:

$SPI = 5.25 \text{ MHz} \rightarrow \text{DMA line time} \approx 60.952 \mu s$

$SPI = 6.00 \text{ MHz} \rightarrow \text{DMA line time} \approx 53.333 \mu s$

$SPI = 6.30 \text{ MHz} \rightarrow \text{DMA line time} \approx 50.794 \mu s$

$SPI = 7.00 \text{ MHz} \rightarrow \text{DMA line time} \approx 45.714 \mu s$

$SPI = 8.00 \text{ MHz} \rightarrow \text{DMA line time} \approx 40.000 \mu s$

The selected clock must also satisfy the actual SPI peripheral clock/divider configuration and the electrical limits of the output path.

7. DMA Double Buffering

There are two different concepts that should not be confused: framebuffer double buffering and DMA double buffering.

Mechanism	Size	Purpose
Framebuffer A	8,000 B	Complete 320×200 image
Framebuffer B	8,000 B	Second complete image
DMA line buffer A	40 B	Current/next line transfer
DMA line buffer B	40 B	Alternate line transfer

If the project uses DMA double-buffer mode with two 40-byte line buffers, the DMA engine can transfer one buffer while software prepares the other. If the project instead uses two complete 8,000-byte framebuffers, rendering can occur into the back framebuffer while the video engine scans the front framebuffer.

8. Framebuffer Double-Buffer Mathematics

Front buffer = buffer currently being displayed

Back buffer = buffer currently being rendered

At a safe frame boundary: swap(front, back)

Memory required = $2 \times \text{WIDTH} \times \text{HEIGHT} / 8$

For 320×200: $2 \times 320 \times 200 / 8 = 16,000$ bytes

The important synchronization rule is that the pointer swap must happen at a frame-safe boundary. Swapping while the display is reading the middle of a frame can produce tearing or a mixture of two images.

9. DMA Line Address Mathematics

$\text{LINE_STRIDE} = \text{WIDTH} / 8 = 40$ bytes

$\text{line_address}(y) = \text{framebuffer} + y \times \text{LINE_STRIDE}$

$\text{line_address}(0) = \text{framebuffer} + 0$

$\text{line_address}(1) = \text{framebuffer} + 40$

$\text{line_address}(199) = \text{framebuffer} + 199 \times 40 = \text{framebuffer} + 7,960$

Last byte of the 200-line framebuffer = offset 7,999

This 40-byte stride is essential. Using 320 as the byte stride would incorrectly treat the pixel width as a byte width and would overrun the framebuffer.

10. DMA Transfer Timing per Line

For a line y , DMA reads exactly 40 bytes:

$\text{DMA_y} = \text{framebuffer}[y \times 40 \dots y \times 40 + 39]$

Bytes/line = 40

Bits/line = 320

$\text{DMA time} = 320 / f_{\text{SPI}}$

Available active-video time $\approx 52.7 \mu\text{s}$

Timing margin = $T_{\text{ACTIVE}} - T_{\text{DMA}}$

A positive timing margin means the transfer finishes before the active-video interval ends. A negative margin means the SPI stream is too slow for the chosen active-video window.

11. CPU Bandwidth and DMA Advantage

Video data per frame = 8,000 bytes
At 29.97 frames/s: $8,000 \times 29.97 \approx 239,760$ bytes/s
 ≈ 234.1 KiB/s of framebuffer data scanned per second
Double buffering does not double the scanout bandwidth; it doubles framebuffer storage.

DMA removes the need for the CPU to manually write every SPI byte at pixel time. The CPU can instead prepare the next line or render into the back framebuffer while DMA performs the timed transfer.

12. Complete Line Timing Relationship

The complete line can be expressed as:

$$T_LINE = T_SYNC + T_BACKPORCH + T_DMA_VIDEO + T_FRONT_PORCH + T_MARGIN$$

The ideal design makes T_DMA_VIDEO fit inside the active-video region, rather than allowing DMA timing to determine the horizontal line period.

13. Timer and SPI Are Independent Clocks

This distinction is critical for debugging the project:

Subsystem	Controls
NTSC timer	63.556 μ s horizontal line
Timer compare	Sync pulse placement/width
SPI clock	Pixel-bit serialization speed
DMA	Automatic transfer of pixel bytes
Framebuffer	Image content
Video resistor network	Electrical black/white levels

14. Vertical Timing Mathematics

```
Lines/frame = 525
Lines/field = 525 / 2 = 262.5
Field period = 262.5 × 63.5556 μs ≈ 16.6833 ms
Frame period = 2 × 16.6833 ms ≈ 33.3667 ms
Frame rate ≈ 29.97 Hz
Field rate ≈ 59.94 Hz
```

The vertical sync pattern is more complicated than one long pulse. Equalizing and serration pulses and the half-line relationship are part of receiver-compatible NTSC interlacing. The supplied source explicitly treats this as a timing state machine driven from the horizontal-rate timing.

15. Mapping 200 Software Rows into the NTSC Raster

The project has 200 logical image rows while the NTSC raster has approximately 480 visible lines. Therefore a mapping is required between the software framebuffer row and the transmitted raster line.

```
y_fb = floor((y_video - Y_START) × 200 / H_VISIBLE)
```

This is a scaling/mapping equation, not a requirement of NTSC itself. If each framebuffer row is intended to occupy one raster line, then the remaining visible raster lines are simply unused or filled according to the project's chosen vertical layout.

16. Horizontal Pixel Mapping for 320 Pixels

```
x = 0 ... 319
byte_index = x >> 3 = floor(x / 8)
bit_index = x & 7
byte_address = line_base + byte_index
pixel_width = T_ACTIVE / 320
```

With $T_{\text{ACTIVE}} \approx 52.7 \mu\text{s}$:

```
T_PIXEL ≈ 52.7 μs / 320 ≈ 164.69 ns
Logical pixel rate ≈ 1 / 164.69 ns ≈ 6.072 MHz
```

Because one SPI bit represents one monochrome pixel, the required SPI bit clock is approximately the same 6.072 MHz value.

This differs from a design where one transmitted byte represents a larger logical pixel group. In this project, 320 pixels are represented by 320 serial bits per line, so the SPI bit rate is the pixel/sample rate.

17. Important Correction to the Original Mathematics

The supplied document used examples such as 64 pixels at roughly 1.214 MHz and discussed CPU/timer pixel output. Those calculations are useful for a general non-DMA design, but they are not the correct final model for the project's 320×200 SPI-DMA scanout.

For the actual 320×200 SPI-DMA design:

```
320 pixels/line
40 bytes/line
320 SPI bits/line
```

SPI $\approx$ 6.07 MHz for a 52.7 μ s active interval

Horizontal timer $\approx$ 15.734 kHz

Timer line period $\approx$ 63.556 μ s

18. Practical STM32F407 DMA Design Equations

For a DMA line engine, the conceptual sequence is:

1. Timer reaches active-video start.
2. Enable/start SPI DMA transfer.
3. DMA transfers 40 bytes.
4. SPI shifts 320 bits.
5. DMA transfer completes.
6. Active video ends.
7. Timer generates the next horizontal sync.
8. Select the next framebuffer row.
9. Repeat.

The next row address is:

`next_line = current_line + 40`

At the end of the image:

`line 199 offset = 199  $\times$  40 = 7,960 bytes`

19. Timing Margin and Design Validation

The most useful hardware test is to measure the actual waveform rather than relying only on calculated values.

Measured line period should be near 63.556 μ s

Measured horizontal sync should be near 4.7 μ s for the chosen approximation

SPI DMA transfer should fit inside active video

DMA completion must occur before the output needs the next timing phase

Framebuffer addresses must remain within offsets 0...7,999

20. Final Project Mathematics Summary

Quantity	Equation	Project value
Frame rate	f_F	29.97 Hz
Frame period	$1/f_F$	33.3667 ms
Lines/frame	N_L	525
Horizontal rate	525×29.97	15.734 kHz
Line period	$1/f_H$	63.5556 μ s
Timer clock	f_{TIM}	84 MHz example
Timer tick	$1/f_{TIM}$	11.90476 ns
Timer counts/line	$T_H \times f_{TIM}$	$\approx$ 5,340
Sync counts	$4.7 \mu s \times 84 \text{ MHz}$	$\approx$ 395
Pixels/line	WIDTH	320
Bytes/line	WIDTH/8	40
Bits/line	40×8	320
Framebuffer	40×200	8,000 B

Quantity	Equation	Project value
Double framebuffer	$2 \times 8,000$	16,000 B
Active time	T_{ACTIVE}	$\approx 52.7 \mu\text{s}$
Required SPI	$320 / T_{\text{ACTIVE}}$	$\approx 6.07 \text{ MHz}$
DMA line time	$320 / f_{\text{SPI}}$	$\approx 52.7 \mu\text{s} @ 6.07 \text{ MHz}$

Bottom line: the horizontal timer owns NTSC timing; SPI owns pixel serialization; DMA owns the movement of the 40-byte line; the framebuffer owns the image; and double buffering separates rendering from scanout. The central design equation is $f_{\text{SPI}} = (\text{WIDTH} / 8 \times 8) / T_{\text{ACTIVE}} = \text{WIDTH} / T_{\text{ACTIVE}}$.

Source basis: supplied document “NTSC Monochrome Composite Video — Mathematics and STM32F407 Timing,” adapted here specifically for the project's SPI DMA and double-buffer architecture.